Forex Automated Trading Portal – Requirements Specification

Version 1.0 · Prepared October 23, 2026 · Intended as the master brief for AI-assisted code generation

1. Project Overview

Product name (working): ForexPilot (rename freely).

Purpose: A web-based portal and trading platform for forex that connects to an Exness MetaTrader 5 (MT5) account, streams live prices, downloads and stores historical data, backtests strategies (including strategies converted from TradingView Pine Script), generates detailed reports, and runs strategies automatically with strict risk controls.

Primary goals

1. Connect to Exness via MT5 and show live prices.
2. Download, store and manage historical market data.
3. Backtest strategies realistically (spread, commission, swap, slippage) and produce detailed reports.
4. Let the user paste open-source Pine Script and get it converted to a runnable Python strategy, then backtested.
5. Run strategies automatically on a demo account first, and on a live account only after explicit opt-in.
6. Provide a polished, professional, dark-themed web interface.

2. Constraints and Assumptions

- **Budget: zero.** Use only free tiers and free open-source software. No paid servers, APIs or subscriptions.
- **Local-first.** Phase 1 runs entirely on the user's Windows laptop. Production hosting is deferred.
- **Demo-first.** All trading and automation targets an Exness **demo** account until the user explicitly enables live mode.
- **Operating system:** Windows 10/11 (required for the MetaTrader5 Python library).

- **Already installed:** MT5, Python, and an Exness account. Supabase and Vercel accounts exist.
- **Single primary user** in the local phase; multi-user support is a later phase.
- **Exness connectivity:** via the MT5 terminal (login number, trading password, server name), not a REST API key.
- **Disclaimer:** the software is a research and automation tool, not financial advice. Past backtest performance does not guarantee future results.

3. Users and Roles

Role	Description	Phase
Owner/Trader	Single user who configures broker, data, strategies and runs bots	1
Admin	Manages users, system settings, audit log	3 (production)
Standard user	Own accounts, strategies, backtests	3 (production)

4. Technology Stack

Layer	Choice	Notes
Frontend	Next.js (App Router) + TypeScript + Tailwind CSS + shadcn/ui	Deployable to Vercel unchanged
Charts	TradingView Lightweight Charts (price); Recharts or Apache ECharts (equity, drawdown, heatmaps)	Free
Backend	Python 3.11/3.12 + FastAPI + Uvicorn	Runs locally
Broker link	`MetaTrader5` Python package (Windows)	Requires MT5 terminal running and logged in
Data processing	pandas, numpy, pyarrow, DuckDB (optional)	Parquet storage
Backtesting	Custom event-driven engine (primary); vectorbt or backtesting.py optional	Must model costs

Layer	Choice	Notes
Metrics and reports	pandas, numpy, Jinja2 templates → HTML; optional PDF via browser print or WeasyPrint	
Realtime	WebSocket from FastAPI to the browser	Live ticks, bot status
Task queue	Python background workers (asyncio / `concurrent.futures`) in Phase 1; Celery + Redis optional later	Avoid extra infra locally
Auth and cloud DB	Supabase (Auth + Postgres + Row Level Security)	Optional in Phase 1, required in production
Local DB (Phase 1)	SQLite	Zero setup
Source control	GitHub (private repo)	
Frontend hosting	Vercel (free plan)	Later phase
AI features	Gemini API free tier (or similar), server-side only	Pine conversion and report explanations
Alerts	Telegram Bot API (free)	Optional

5. Architecture

5.1 Local phase (Phase 1)

```
Browser (localhost:3000, Next.js)
   ⇅ REST + WebSocket
FastAPI backend (localhost:8000)
   ├── MT5 service        → MT5 terminal → Exness demo account
   ├── Data service       → Parquet files in /data
   ├── Backtest service   → engine + report generator
   ├── Strategy runner    → paper/live execution + risk manager
   ├── Pine converter     → LLM call → generated Python strategy
   └── SQLite             → metadata, runs, trades, settings
```

5.2 Production phase (later)

- **Vercel:** Next.js website only.
- **Supabase:** auth, users, saved strategies, backtest results, settings, audit log.
- **Always-on engine host (future decision):** runs the MT5 connection, live feed, strategy runner and scheduled data updates.
- **Communication:** the website and engine do not call each other directly. They exchange state through Supabase (commands such as "start strategy" written by the website, status and trades written by the engine), with Supabase Realtime for live updates.
- **Large data:** Parquet files on the engine host, optionally backed up to free object storage.

6. Integration Requirements

6.1 Exness / MetaTrader 5

- Connect using `mt5.initialize(login, password, server)` with credentials from environment variables (never hard-coded, never sent to the browser).
- Support Exness symbol naming differences: standard accounts typically use a suffix (e.g. `EURUSDm`, `XAUUSDm`), while raw/zero accounts typically do not. Provide a **symbol mapping/resolver** configurable per account.
- Required operations: account info, symbol list and specs (digits, point, contract size, min/max/step volume, stops level), live tick/quote, historical candles (`copy_rates_range`, `copy_rates_from_pos`), historical ticks (`copy_ticks_range`) where available, open positions, pending orders, order send/modify/close, trade history.
- Detect and report: terminal not running, not logged in, algo trading disabled, market closed, invalid stops, insufficient margin, requote/off-quotes.
- Auto-reconnect with exponential backoff; emit a "disconnected" alert.
- Note that available history depth depends on the broker server; the data manager must report the actual range obtained.

6.2 Supabase

- Auth: email/password and optional Google login.
- Postgres tables with Row Level Security on every table, scoped by `user_id`.
- Storage bucket for small files (reports, exported strategies); not for bulk candle data.

- Use the anon key in the frontend only; the service role key stays on the backend and never reaches the browser or GitHub.

6.3 Vercel and GitHub

- GitHub private repo (monorepo: `/backend`, `/frontend`, `/docs`).
- `.gitignore` must exclude `.env`, `.venv`, `data/`, `node_modules`, `__pycache__`.
- Vercel deploys `/frontend` with environment variables set in the dashboard.

6.4 AI (Gemini or equivalent)

- Used for Pine Script → Python conversion and optional natural-language report summaries.
- API key kept server-side in `.env`.
- Provide a clean provider interface so the model can be swapped.

6.5 Telegram (optional)

- Alerts for order opened/closed, error, disconnect, risk limit hit, daily summary.

7. Functional Requirements

Priority: **M** = must, **S** = should, **C** = could.

7.1 Authentication and Account Management

- FR-A1 (M): Local mode with no login in Phase 1; login screen via Supabase in production mode.
- FR-A2 (M): Broker connection settings page (login, server, symbol suffix) with a "Test connection" button and clear status.
- FR-A3 (M): Credentials stored encrypted at rest (or in `.env` locally); never displayed after saving.
- FR-A4 (S): Support multiple broker accounts, each labelled DEMO or LIVE.
- FR-A5 (S): Audit log of logins, settings changes and every automated order.

7.2 Dashboard

- FR-D1 (M): KPI cards: balance, equity, margin, free margin, open P&L, today's P&L, connection status, active bots.

- FR-D2 (M): Persistent LIVE/DEMO badge in the header on every page.
- FR-D3 (M): Open positions table with close/modify actions and pending orders.
- FR-D4 (S): Equity curve (session and historical) and recent activity feed.

7.3 Live Market Data

- FR-M1 (M): Watchlist with live bid/ask/spread, change and flash on tick.
- FR-M2 (M): Interactive candlestick chart with timeframes M1–D1, zoom/scroll, and live candle update.
- FR-M3 (S): Indicator overlays (SMA, EMA, Bollinger Bands, RSI, MACD, ATR).
- FR-M4 (S): Drawing tools (trend line, horizontal line).
- FR-M5 (C): Economic news calendar and news blackout flags.

7.4 Historical Data Manager

- FR-H1 (M): Select symbol(s), timeframe(s) and date range, then start a download job with progress.
- FR-H2 (M): Save as Parquet in `data/{symbol}/{timeframe}/{year}.parquet` (partitioned by year).
- FR-H3 (M): Incremental update (append only new candles) and manual "update all".
- FR-H4 (M): Data catalog page: symbol, timeframe, first/last candle, row count, file size, last updated, quality status.
- FR-H5 (M): Data quality checks: duplicates, gaps (excluding weekends and holidays), invalid OHLC (high < low, etc.), zero-volume anomalies; show a report and offer auto-fix.
- FR-H6 (S): Import CSV (e.g. from other data providers) with column mapping and timezone selection.
- FR-H7 (S): Export to CSV/Parquet.
- FR-H8 (S): Tick data download for selected short periods where the broker provides it.
- FR-H9 (M): Store all timestamps in UTC; record the broker server time offset.

7.5 Strategy Management

- FR-S1 (M): Strategy library listing name, type (Python / converted from Pine / no-code), version, tags, last backtest result.
- FR-S2 (M): Code editor (Monaco) for Python strategies with syntax highlighting and a template.

- FR-S3 (M): Strategy interface (see 11.1) with typed parameters that auto-generate the parameter form in the UI.
- FR-S4 (S): Version history with diff and restore.
- FR-S5 (S): No-code builder: choose indicators, define entry/exit conditions, stop-loss/take-profit and position sizing.
- FR-S6 (S): Built-in example strategies: SMA crossover, RSI mean reversion, Bollinger breakout, Donchian breakout.
- FR-S7 (M): Strategies run in a restricted sandbox (no file system, network or `os` access) because they may be AI-generated or pasted.

7.6 Backtesting

- FR-B1 (M): Configure a run: strategy, parameters, symbol, timeframe, date range, initial balance, leverage, position sizing, and cost model.
- FR-B2 (M): **Cost model:** spread (fixed, or from historical bar/tick spread), commission per lot, slippage (fixed or random), swap/rollover (long/short per day).
- FR-B3 (M): Fill logic: signal on bar close → fill at next bar open (configurable); intrabar stop-loss/take-profit resolution with conservative handling when both are hit in one bar.
- FR-B4 (M): Support long, short, multiple concurrent positions (optional), partial closes, trailing stops, and break-even logic.
- FR-B5 (M): No look-ahead: indicators computed only from data available at each bar; the engine must guard against future data access.
- FR-B6 (M): Run in a background job with progress, cancel, and result persistence.
- FR-B7 (M): Lot sizing and P&L conversion handled correctly for the account currency, including JPY pairs and gold.
- FR-B8 (S): Parameter optimization: grid and random search with parallel workers, results table and 2D heatmap.
- FR-B9 (S): Walk-forward analysis and in-sample/out-of-sample split.
- FR-B10 (S): Monte Carlo (trade reshuffling and resampling) with confidence bands for equity and drawdown.
- FR-B11 (S): Multi-symbol portfolio backtest.
- FR-B12 (C): Compare multiple runs side by side.

7.7 Reports

Every backtest produces a detailed report (web view plus downloadable HTML and PDF) containing:

1. **Summary:** strategy, parameters, symbol, timeframe, period, initial/final balance, net profit, return %, CAGR.
2. **Performance metrics:** profit factor, expectancy, win rate, average win/loss, payoff ratio, largest win/loss, max consecutive wins/losses, Sharpe, Sortino, Calmar, recovery factor, exposure time, trades per month.
3. **Risk metrics:** max drawdown (amount, %, duration), average drawdown, Ulcer index, value at risk, margin usage.
4. **Charts:** equity curve, drawdown (underwater) chart, monthly returns heatmap, return distribution histogram, P&L by hour of day / day of week, trade P&L scatter, MAE/MFE.
5. **Trade list:** entry/exit time and price, direction, size, SL/TP, P&L, pips, duration, costs; sortable, filterable, exportable.
6. **Chart with trades:** price chart with entry/exit markers.
7. **Costs breakdown:** total spread, commission, swap, slippage.
8. **Robustness section:** out-of-sample result, Monte Carlo summary, parameter sensitivity, and an **overfitting warning score** with plain-language explanation.
9. **Assumptions and warnings:** data source, data quality notes, cost assumptions, and any look-ahead/repainting warnings.
10. **Optional AI summary:** plain-language interpretation of strengths and weaknesses (clearly labelled as AI-generated).

7.8 Pine Script Import

- FR-P1 (M): A page where the user pastes Pine Script (v4/v5/v6) or a TradingView script URL's code, with optional notes.
- FR-P2 (M): The backend sends the code to the AI provider with a strict prompt and returns: (a) a Python strategy conforming to the strategy interface, (b) a list of detected inputs mapped to UI parameters, (c) warnings for unsupported or ambiguous constructs.
- FR-P3 (M): Show Pine and generated Python side by side for review; the user must approve before saving.
- FR-P4 (M): Static checks on generated code (syntax, forbidden imports, sandbox rules) and an automatic smoke test on a small data sample.

- FR-P5 (M): Detect and warn about: `request.security` / `lookahead`, repainting patterns, `barstate` logic, unsupported built-ins, and drawing-only indicators with no `strategy.*` calls.
- FR-P6 (M): Convert Pine `strategy()` settings (commission, slippage, pyramiding, default quantity, process_orders_on_close) into backtest config.
- FR-P7 (S): Indicator-only scripts: ask the user to define entry/exit rules, or let the AI propose rules for approval.
- FR-P8 (S): Verification helper: let the user upload a TradingView "List of trades" CSV export and compare it with the portal's trades, highlighting differences in count, timing and P&L.
- FR-P9 (M): Store the source script, its license text/author/URL if provided, and show a reminder to respect the script's license.
- FR-P10 (C): Library of reusable translated Pine built-ins (ta.sma, ta.ema, ta.rsi, ta.atr, ta.crossover, etc.) implemented and unit-tested in Python so the AI reuses them rather than re-inventing them.

7.9 Automated Trading

- FR-T1 (M): Modes: **Backtest**, **Paper** (simulated fills on live prices), **Demo-live** (real orders on demo account), **Live** (real orders on a live account, off by default).
- FR-T2 (M): Start/stop/pause a bot per strategy, symbol, timeframe and account; show status, last signal, open trades and logs.
- FR-T3 (M): Strategy runner evaluates on new closed bars (and optionally ticks), with idempotent order handling to avoid duplicate orders after reconnects.
- FR-T4 (M): **Risk manager** applied to every order: max risk per trade (% of equity), max lot size, max open positions (total and per symbol), max daily loss, max drawdown, max spread filter, trading-hours filter, news blackout, and allowed symbols.
- FR-T5 (M): **Kill switch:** one-click "Close all and stop everything" available on every page.
- FR-T6 (M): Always attach stop-loss (and take-profit where defined) to orders so positions are protected at the broker if the engine disconnects.
- FR-T7 (M): Switching a bot to Live mode requires a confirmation dialog stating the account number, symbol and risk limits, and a typed confirmation.
- FR-T8 (M): Persist bot state so a restart resumes safely: reconcile open positions with MT5 before acting.
- FR-T9 (S): Alerts via Telegram/email for key events.

- FR-T10 (S): Daily and weekly performance summaries.
- FR-T11 (S): Compare live results against the backtest expectation and flag degradation.
- FR-T12 (M): Every automated order is written to the audit log with the strategy, signal reason and risk-check outcome.

7.10 Trade Journal and Analytics

- FR-J1 (S): Journal entries linked to trades with notes, tags and screenshots.
- FR-J2 (S): Performance analytics across live and demo trades: by strategy, symbol, hour, weekday, and tag.
- FR-J3 (C): AI assistant chat that answers questions about backtests and live results using the user's own data.

7.11 Settings

- FR-X1 (M): Broker accounts, symbol mapping, timezone, base currency, default cost model.
- FR-X2 (M): Risk defaults, kill-switch behaviour, alert channels.
- FR-X3 (S): Theme (dark default, light option), number and date formats.
- FR-X4 (S): Data folder location and backup/export of all settings and strategies.

8. Data Requirements

8.1 Candle storage (Parquet)

- Path: `data/{SYMBOL}/{TIMEFRAME}/{YEAR}.parquet`.
- Columns: `time` (UTC), `open`, `high`, `low`, `close`, `tick_volume`, `spread`, `real_volume`.
- Sorted by time, unique on `time`, validated on write.
- Tick files (optional): `data/{SYMBOL}/ticks/{YYYY-MM}.parquet` with `time_msc`, `bid`, `ask`, `last`, `volume`, `flags`.

8.2 Relational schema (SQLite locally, Postgres on Supabase)

All tables carry `id`, `created_at`, and (in Supabase) `user_id` with Row Level Security.

- `broker_accounts`: label, broker, server, login, mode (DEMO/LIVE), symbol_suffix, encrypted_credentials_ref, is_active.

- `symbols`: broker_account_id, name, digits, point, contract_size, volume_min/max/step, currency_profit, stops_level.
- `datasets`: symbol, timeframe, first_time, last_time, row_count, file_path, quality_status, last_updated.
- `data_jobs`: type, params_json, status, progress, error, started_at, finished_at.
- `strategies`: name, description, source_type (python/pine/nocode), code, pine_source, params_schema_json, version, parent_id, license_note, tags.
- `backtest_runs`: strategy_id, params_json, symbol, timeframe, start, end, cost_model_json, status, metrics_json, report_path, equity_curve_ref.
- `backtest_trades`: run_id, entry/exit time and price, direction, size, sl, tp, pnl, pips, costs, mae, mfe.
- `optimization_runs`: strategy_id, param_grid_json, status, results_ref.
- `bots`: strategy_id, broker_account_id, symbol, timeframe, mode, params_json, status, risk_config_json, last_heartbeat.
- `orders` and `trades_live`: bot_id, mt5_ticket, side, volume, price, sl, tp, status, reason, risk_check_json, opened_at, closed_at, pnl.
- `risk_events`: bot_id, type, details, created_at.
- `journal_entries`: trade_id, note, tags, attachment_ref.
- `audit_log`: actor, action, entity, details_json, created_at.
- `settings`: key, value_json.

9. Backend API (FastAPI)

REST under `/api/v1`, WebSocket under `/ws`. All responses typed with Pydantic models; auto-generated OpenAPI docs.

Area	Endpoints (examples)
Health/Broker	`GET /health`, `POST /broker/connect`, `GET /broker/status`, `GET /broker/account`
Market	`GET /symbols`, `GET /symbols/{s}/spec`, `GET /candles?symbol&tf&from&to`, `WS /ws/ticks?symbols=`

Area	Endpoints (examples)
Data	`POST /data/jobs`, `GET /data/jobs/{id}`, `GET /data/catalog`, `POST /data/quality-check`, `POST /data/import`, `GET /data/export`
Strategies	`GET/POST /strategies`, `GET/PUT/DELETE /strategies/{id}`, `POST /strategies/validate`
Pine	`POST /pine/convert`, `POST /pine/verify-trades`
Backtests	`POST /backtests`, `GET /backtests/{id}`, `GET /backtests/{id}/trades`, `GET /backtests/{id}/report`, `POST /backtests/{id}/cancel`
Optimization	`POST /optimizations`, `GET /optimizations/{id}`
Bots	`GET/POST /bots`, `POST /bots/{id}/start
Trading	`GET /positions`, `POST /orders`, `POST /positions/{ticket}/close`, `POST /kill-switch`
Risk/Settings	`GET/PUT /risk`, `GET/PUT /settings`, `GET /audit-log`

10. UI/UX Requirements

- **Look and feel:** modern, clean, professional; dark theme by default with a light option; one accent colour; green/red reserved for profit/loss; generous spacing; subtle animation; skeleton loaders; clear empty and error states.
- **Typography:** Inter (or similar) for text and a monospace face with tabular numbers for prices and P&L.
- **Layout:** left sidebar navigation (Dashboard, Markets, Data, Strategies, Pine Import, Backtests, Optimization, Bots, Positions, Journal, Reports, Settings), top bar with account switcher, connection indicator, LIVE/DEMO badge and kill switch.
- **Responsive:** fully usable on desktop and tablet; read-only monitoring on mobile (PWA optional).
- **Charts:** smooth, zoomable, synchronized crosshair where relevant; consistent colours.

- **Accessibility:** keyboard navigation, sufficient contrast, labels on all controls.
- **Marketing site (production phase):** landing page with hero, feature sections, product screenshots, FAQ, risk disclaimer, pricing placeholder, privacy and terms pages.
- **Safety UX:** destructive and live-trading actions require confirmation; risk disclaimer shown on first launch and in the footer.

11. Strategy Interface and Engine Rules

11.1 Python strategy contract

A strategy is a class with:

- `name`, `version`, and a `params` schema (name, type, default, min, max, step, description).
- `init(self, ctx)`: declare indicators and warm-up period.
- `on_bar(self, ctx, bar)`: called once per closed bar; may call `ctx.buy()`, `ctx.sell()`, `ctx.close()`, `ctx.modify()` with size, stop-loss, take-profit and comment.
- Optional `on_tick` for live use only.
- Access to history via `ctx.data` that is **sliced to the current bar** to prevent look-ahead.
- The same strategy code must run unchanged in Backtest, Paper and Live modes (the engine swaps the execution adapter).

11.2 Engine rules

- Deterministic: same data and parameters give the same result.
- Time handling in UTC; sessions and filters expressed in a configured timezone.
- Position sizing options: fixed lot, fixed risk %, fixed fractional, volatility-based.
- Unit-tested against hand-calculated examples for P&L, pip value, swaps and drawdown.

12. Non-Functional Requirements

- **Security:** secrets in environment variables; encrypted credential storage; no secrets in the frontend, logs or GitHub; CORS restricted; input validation on all endpoints; sandboxed strategy execution; Row Level Security on every Supabase table; rate limiting in production.

- **Reliability:** graceful handling of MT5 disconnects; idempotent order submission; heartbeat and watchdog; safe restart and position reconciliation; structured logging with rotation.
- **Performance:** a 5-year H1 backtest completes in seconds; a 1-year M1 backtest in under a minute on a typical laptop; UI remains responsive during long jobs (background execution with progress).
- **Scalability path:** modules are separated so the engine can later move to an always-on server and the website to Vercel without rewrites.
- **Testing:** unit tests for the engine and indicators, integration tests with mocked MT5, a fixture dataset for reproducible backtests, and a documented manual test checklist for the MT5 connection.
- **Observability:** logs page in the UI; error reporting; health endpoint.
- **Documentation:** README with setup steps, `.env.example`, architecture notes, strategy-writing guide, and a troubleshooting section.
- **Compliance:** visible risk disclaimers; no performance guarantees; note that offering automated trading to other people may require regulatory licensing depending on jurisdiction.

13. Suggested Project Structure

```
forex-portal/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/            (routers)
│   │   ├── core/           (config, security, logging)
│   │   ├── brokers/        (mt5_client.py, symbol_resolver.py)
│   │   ├── data/           (downloader.py, catalog.py, quality.py, store.py)
│   │   ├── engine/         (backtester.py, costs.py, sizing.py, metrics.py,
montecarlo.py)
│   │   ├── strategies/     (base.py, examples/, sandbox.py)
│   │   ├── pine/           (converter.py, prompts/, ta_library.py,
verifier.py)
│   │   ├── trading/        (runner.py, risk_manager.py, executor.py,
reconcile.py)
│   │   ├── reports/        (builder.py, templates/)
│   │   ├── models/         (pydantic + db models)
│   │   └── alerts/         (telegram.py)
```

```
|   ├── tests/
|   ├── requirements.txt
|   └── .env.example
├── frontend/               (Next.js app, components, charts, pages)
├── data/                   (Parquet files, git-ignored)
├── docs/
└── README.md
```

14. Delivery Phases and Acceptance Criteria

Phase	Scope	Acceptance criteria
0. Setup	Repo, environment, MT5 demo, `.env`	`connect_test.py` prints account info and a live price
1. Data	Downloader, Parquet store, catalog, quality checks	Download 5 years of H1 for 3 symbols; catalog shows correct ranges; quality report runs
2. Backtester and reports	Engine, cost model, metrics, HTML report, example strategies	Known-answer unit tests pass; SMA crossover report generated with all report sections
3. Dashboard	Next.js UI: dashboard, markets, data manager, strategies, backtests, reports	Full flow from the browser: download → backtest → view report
4. Pine import	Converter, review screen, warnings, verification helper	At least 3 sample open-source Pine strategies convert, run, and show warnings correctly
5. Optimization and robustness	Grid/random search, walk-forward, Monte Carlo, overfitting score	Heatmap and robustness section appear in reports
6. Paper and demo trading	Bot runner, risk manager, kill switch, logs, alerts	Bot runs for 1+ week on demo with no duplicate orders and correct reconciliation after restart
7. Production prep	Supabase auth and RLS, Vercel deploy, hosting decision, landing page	Multi-user login works; data isolated per user; deployed site loads

Phase	Scope	Acceptance criteria
8. Live pilot	Live mode with small size and attended monitoring	Risk limits enforced; kill switch tested; audit log complete

15. Out of Scope for Version 1

- Trading for third-party clients or copy-trading services.
- Native mobile apps.
- Brokers other than Exness (the broker layer should be abstracted so others can be added).
- High-frequency or latency-sensitive strategies.
- Guaranteeing identical results to TradingView's strategy tester.

16. Risks and Mitigations

Risk	Mitigation
Backtest looks great but strategy fails live (overfitting, repainting, look-ahead)	Out-of-sample and walk-forward tests, Monte Carlo, warnings, paper trading before live
Laptop sleeps or disconnects during automated trading	Broker-side SL/TP on every order, demo-only until a stable host exists, reconciliation on restart
AI-converted Pine code is wrong	Mandatory human review, smoke tests, trade-list comparison with TradingView export
Limited broker history depth	Show actual range obtained; allow CSV import from other sources
Duplicate or runaway orders	Idempotent order IDs, per-bot position limits, global kill switch
Credential leakage	Env vars, encryption, `.gitignore`, no secrets in frontend or logs

Risk	Mitigation
Pine script licensing	Store license/author, show reminder, avoid redistributing others' code
Free-tier limits (Vercel, Supabase, AI API)	Keep heavy work local, store bulk data as files, cache AI results

17. Instructions for the AI Code Generator

When generating code from this specification:

1. Build in the phase order above; complete and test each phase before starting the next.
2. Prefer simple, readable, well-commented code with type hints; avoid unnecessary dependencies.
3. Write unit tests alongside the engine, metrics and Pine helper functions.
4. Never hard-code credentials; provide `.env.example`.
5. Keep the broker layer behind an interface so the MT5 implementation can be swapped or mocked.
6. Keep one strategy code path for backtest, paper and live modes.
7. Default every trading feature to demo/paper and require explicit opt-in for live.
8. Provide a README with exact Windows setup commands and a troubleshooting section.
9. Flag any part that could not be verified without a live MT5 terminal so the user knows what to test manually.
10. Keep the UI consistent with the design requirements in section 10.

18. Open Decisions (to confirm later)

- Final product name and branding.
- Which AI provider to use for Pine conversion (free-tier limits to be checked).
- Production engine hosting approach once the local version is proven.
- Whether to open the portal to other users or keep it personal.
- Which currency pairs and timeframes to prioritise first (suggested: EURUSD, GBPUSD, USDJPY, XAUUSD on H1 and M15).